LAN & IM Communication SDK Integration Guide

Permission

Mobile App

```
  <uses-permission android:name="android.permission.INTERNET" />
  <uses-permission android:name="android.permission.CHANGE_NETWORK_STATE" />

  <!—Add permission for “create” and “delete” data into SDCard -->
  <uses-permission
android:name="android.permission.MOUNT_UNMOUNT_FILESYSTEMS"/>
  <!-- Add permission for “write” data into SDCard -->
  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
```

Robot App

```
  <uses-permission android:name="com.ubtrobot.master.permission.MASTER" />

  <!-- Add permission for “create” and “delete” data into SDCard -->
  <uses-permission
android:name="android.permission.MOUNT_UNMOUNT_FILESYSTEMS"/>
  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
```

Dependencies

Mobile App

Please add dependencies according to your IM solution provider

Robot App

```
Implement 'com.ubtrobot.communication:communication-robot:0.0.19-SNAPSHOT'
```

Mobile API

CommChannelManager class

Initiate

```
public static void init(Context context);
```

for example:

```
// Initiate in YourApplication.onCreate
public class YourApplication extends Application {

 @Override
 public void onCreate() {
   super.onCreate();
   // Initiate
   CommChannelManager.init(getApplicationContext());
   // After initiated, an instance of CommChannelManager can be got
   CommChannelManager manager = CommChannelManager.getInstance();
}
}
```

Create Channel

```
/**
* Create Channel
* @param channelName
* @return
*/
public Channel createChannel(String channelName);
```

Destroy Channel

```
/**
* Destroy channel
* @param channelName
*/
public void destroyChannel(String channelName);
```

Get Channel by Name

```
/**
* Get channel by name
* @return
*/
public Channel getChannelByName(String channelName);
```

List Connection by Robot SN

```
/**
* get connections by robot SN
* @return
*/
public ArrayList<Connection> getConnectionsByRobotSn(String sn);
```

List Connection by Channel Name

```
/**
* get connections by channel name
* @return
*/
public ArrayList<Connection> getConnectionsByChannelName(String channelName);
```

Create Connection

```
/**
* Create SocketConnection
* @return SocketConnection
```

```
*/
protected synchronized SocketConnection createSocketConnection();

/**
* Create IM connection
* @return ImConnection
*/
protected synchronized ImConnection createImConnection();
```

Connection Interface

Get Connection Type

```
/**
* Get connection type
* @return     SOCKET_TYPE | IM_TYPE
*/
ConnectionType connetionType = mConnection.getConnectionType();
```

Connect

```
public void connect(Parameter parameter, @Nullable ConnectionCallback callback);
```

LAN Connect

For example:

```
mLanConnection.connect(new SocketConnectParam(new InetSocketAddress("127.0.0.1",
                                9000)),
                        new ConnectionCallback() {
                            @Override
                            public void onSuccess() {
                                Log.i(TAG, "connect success");
                            }

                            @Override
                            public void onFail(@NonNull NetException ex) {
                                Log.i(TAG, "connect fail", ex);
```

```
                    }
                });
```

IM Connect

For example:

```
mImConnection.connect(
        new ImConnectionParam()
            .setProject("BETA")
            .setFrom(from)
            .setTo(to),
        new ConnectionCallback() {
            @Override
            public void onSuccess() {
            Log.d(TAG, " connect onSuccess");
            }

            @Override
            public void onFail(NetException ex) {
                Log.d(TAG,
                " connect onFail:" + ex.getCode() + " description:" + ex.getMessage());
            }
});
```

Disconnect

```
public void disconnect(@Nullable ConnectionCallback callback);
```

For example:

```
//LAN
mConnection.disconnect(new ConnectionCallback() {
                    @Override
                    public void onSuccess() {
                        Log.d(TAG, "disconnect success");
                    }

                    @Override
```

```
            public void onFail(@NonNull NetException exception) {
                Log.e(TAG, "disconnect fail");
            }
        });
```

Register Connection State Listener

```
public synchronized void registerConnectionStateListener(@NonNull
ConnectionStateListener listener);
```

For example:

```
ConnectionStateListener mConnectionStateListener = new ConnectionStateListener() {
        @Override
        public void onStateChanged(INet.ConnectionState newState) {
            Log.d("lan-test-net", " new state:" + newState);
        }
    };


mConnection.registerConnectionStateListener(mConnectionStateListener);
```

Unregister Connection State Listener

```
public synchronized void unregisterConnectionStateListener(@NonNull
ConnectionStateListener listener);
```

For example:

```
mConnection.unregisterConnectionStateListener(mConnectionStateListener);
```

Add Push Message Listener

```
/**
* Add push message listener
* @param listener
```

```
*/
public void addPushMessageListener(PushMessageListener listener);
```

For example:

```
private PushMessageListener mPushMessageListener = new PushMessageListener() {
    @Override
    public void onReceive(PushMessage message) {
        Log.d(TAG, "Recive a push message:" + message);
    }
};

mConnection.addPushMessageListener(mPushMessageListener);
```

Remove Push Message Listener

```
/**
* Remove push message listener
* @param listener
*/
public void removePushMessageListener(PushMessageListener listener);
```

Set Encryption for Data Transfer

```
/**
* Set encryption for data transfer
*
* Currently, only support encryption for LAN communication, if this function is not called and changed encryption method, the default method is one-to-one AES encryption, highly recommended to use default encryption method
*
* @param algorithm value:
EncryptAlgorithm.NONE/EncryptAlgorithm.AES_IJM_CBC_128
*                    /EncryptAlgorithm.AES_IJM_YJYM_128
*
*/
public abstract void setEncryptAlgorithm4TransferData(int algorithm) throws NetException;
```

Release

```
public void release();//release all resource and unregister all listener and responder
```

Add Message Listener for IM Communication

```
/**
* Add message listner for IM communication
* @param receiveListener
*/
public void addMessageListener(@NonNull MessageListener receiveListener);
```

For example:

```
    mImConnection.addMessageListener(new MessageListener() {
        @Override
        public void onReceive(ImMessage message) {
            Log.d(TAG, "Receive message:" + message);
            if (message.getType() == ImMessageType.File) {
                String path = Environment.getExternalStorageDirectory() + "/testcopy";
                Log.d(TAG, "file path:" + path);
                ImFileMessage fileMessage = (ImFileMessage) message;
                fileMessage.getToFile(path, new CommonCallback() {
                    @Override
                    public void onFail(int var1, String var2) {
                        Log.e(TAG, "get file fail!");
                    }

                    @Override
                    public void onSuccess() {
                        Log.i(TAG, "get file success");
                    }
                });
            }
        }
    });
```

Master Callback

Create Master

```
RobotMaster master = CommChannelManager.getInstance().createRobotMaster(Connection connection);
```

Subscribe Event

```
EventSubscriber mEventSubscriber = new EventSubscriber() {
  @Override
  public void onEventPublish(Event event) {
       Log.i(TAG, "Receive event:" + event);
  }
};

master.subscribe(mEventSubscriber, "ubtrobot.event.test");
```

Unsubscribe Event

```
master.unsubscribe(mEventSubscriber);
```

Retrieve Service

```
RobotServiceProxy service = master.getRobotService("motion") ;

service.call(request, ResponseCallback callback);

RobotService with listed interfaces:

Response call(String path) throws CallException; // Synchronous callback

Response call(String path, @Nullable Param param) throws CallException;

Cancelable call(String path, ResponseCallback callback); //Asynchronous callback,cancelable

Cancelable call(String path, @Nullable Param param, @Nullable ResponseCallback callback);
```

Retrieve Robot Skill

```
RobotSkillsProxy mRobotSkillsProxy =
master.createRobotSkillsProxy();

RobotSkillsProxy.call(request, ResponseCallback callback);

RobotSkillsProxy with listed interfaces:

Response call(String path) throws CallException; // Synchronous callback

Response call(String path, @Nullable Param param) throws CallException;

Cancelable call(String path, ResponseCallback callback); //Asynchronous callback,cancelable


Cancelable call(String path, @Nullable Param param, @Nullable ResponseCallback
callback);
```

Notice: Please refer to common_mobile_demo

Robot API

CommChannelManager Class

```
/**
* Initiate
* @param context
*/
public static void init(Context context) ;

/**
* Start SocketService service
*/
public static void startSocketService();

/**
* Start IM service
*/
public static void startImService();
```

```
/**
* Get IM channel
* @return
*/
public static ImChannelHub getImChannel();

/**
* Get SocketChannel
* @return
*/
public static SocketChannelHub getSocketChannel();
```

ImChannelHub

```
/**
* Add IM message listener
* @param listenner
*/
public void addImMessageListenner(ImMessageListenner listenner);

/**
* Remove IM message listener
* @param listenner
*/
public void removeImMessageListenner(ImMessageListenner listenner);


/**
* Get conversation
* @param peer Peer IM account
* @return
*/
public ImConversation getConversation(String peer);
```

ImConversation

```
/**
* Send Message
```

```
* @param message
* @param callback
*/
void sendMessage(ImMessage message, final SendCallback callback);

/**
* Send file
* @param path     file path
* @param fileName file name shown in the receiver
* @param callback
*/
public void sendFile(String path, String fileName, final SendCallback callback);

/**
* Push message to Mobile App
* @param msg
*/
public void push(ProtoParam msg, ResultCallback callback);
```

For example:

```
// Send file via IM
String path = Environment.getExternalStorageDirectory() + "/test";
ImChannelHub imChannelHub = CommChannelManager.getImChannel();
imChannelHub.sendFile("mark-1", path, "test file from robot", new SendCallback() {
  @Override
  public void onSuccess() {
  Log.d(TAG, "Send success");
  }

  @Override
  public void onFail(@NonNull NetException exception) {
  Log.d(TAG, "Send fail", exception);
  }
});
```

SocketChannelHub

```
/**
* Get LAN connection
* @return SocketConnection
```

```
* notice: need time to operate, please do not retrieve in main progress
*/
public static ArrayList<SocketConnection> getSocketConnections();
```

SocketConnection

```
/**
* Push message to mobile App
* @param msg
*/
@Override
public void push(ProtoParam msg, final ResultCallback callback);
```

Notice:

- Request registration is not supported in Mobile App
- “Publish” event is not supported in Mobile App
- File sent by IM must be less than 20M
- Message sent by IM must be less than 9K byte
- Message sent by LAN must be less than 16K byte